

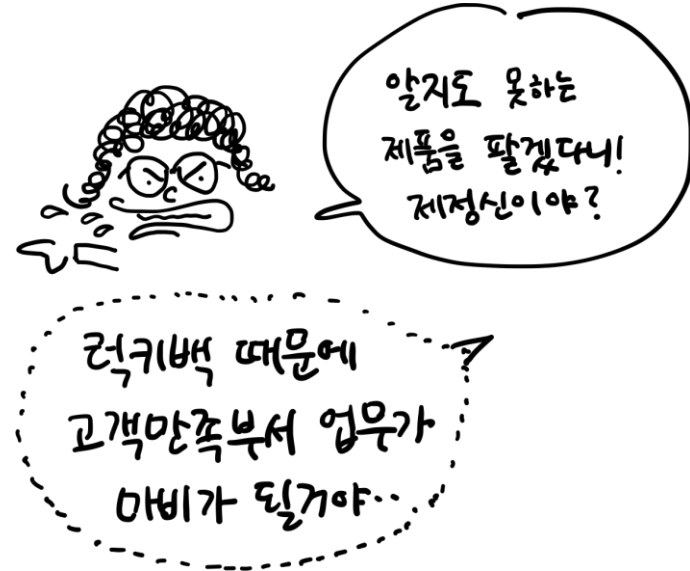


로지스틱 회귀

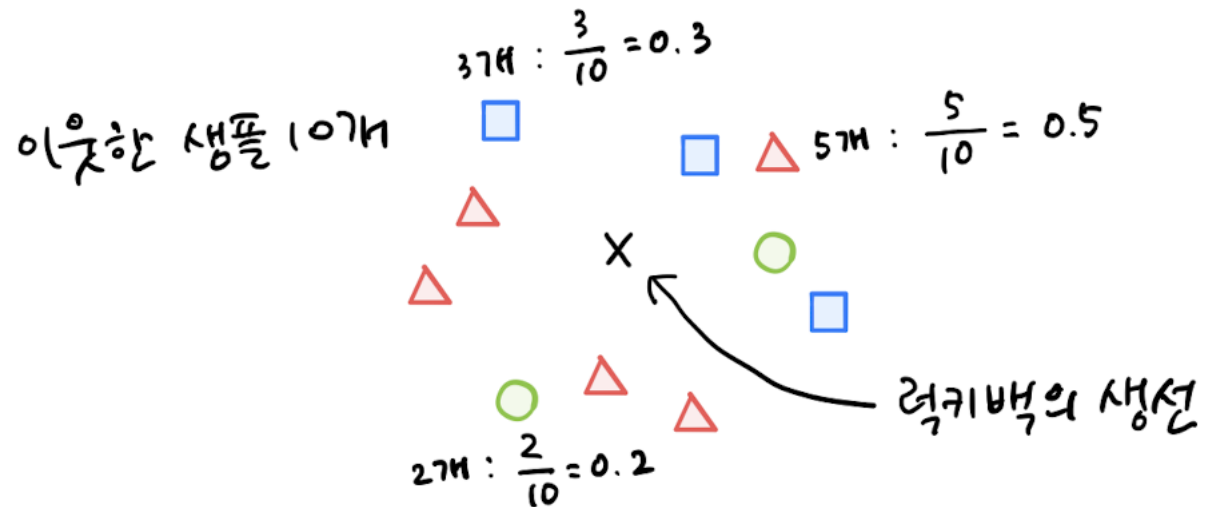
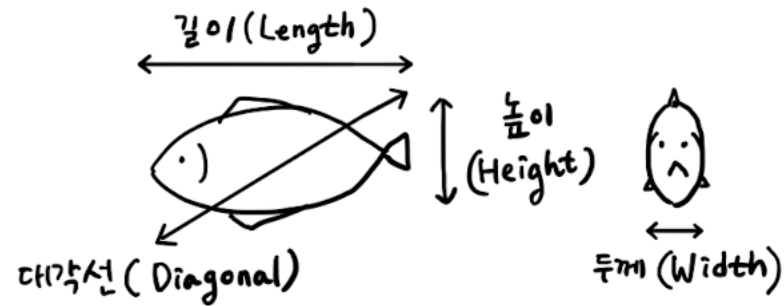
럭키 백



도미일 확률 : 72 %
빙어일 확률 : 16 %
...



확률 계산하기



데이터 준비

```
import pandas as pd
```

```
fish = pd.read_csv('https://bit.ly/fish_csv_data')  
fish.head()
```

	Species	Weight	Length	Diagonal	Height	Width
0	Bream	242.0	25.4	30.0	11.5200	4.0200
1	Bream	290.0	26.3	31.2	12.4800	4.3056
2	Bream	340.0	26.5	31.1	12.3778	4.6961
3	Bream	363.0	29.0	33.5	12.7300	4.4555
4	Bream	430.0	29.0	34.0	12.4440	5.1340

```
fish_input = fish[['Weight', 'Length', 'Diagonal', 'Height', 'Width']].to_numpy()
```

```
fish_target = fish['Species'].to_numpy()
```

k-최근접 이웃의 다중 분류

```
from sklearn.neighbors import KNeighborsClassifier
```

```
kn = KNeighborsClassifier(n_neighbors=3)
kn.fit(train_scaled, train_target)
```

```
print(kn.classes_)
['Bream' 'Parkki' 'Perch' 'Pike' 'Roach' 'Smelt' 'Whitefish']
```

```
print(kn.predict(test_scaled[:5]))
['Perch' 'Smelt' 'Pike' 'Perch' 'Perch']
```

```
proba = kn.predict_proba(test_scaled[:5])
print(np.round(proba, decimals=4))
```

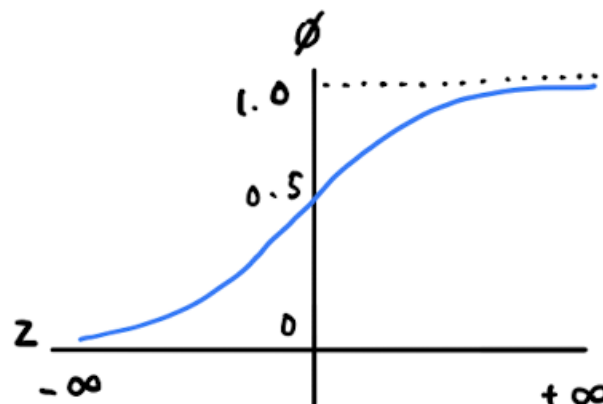
```
[[0. 0. 1. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 1. 0.]
 [0. 0. 0. 1. 0. 0. 0.]
 [0. 0. 0.6667 0. 0.3333 0. 0.]
 [0. 0. 0.6667 0. 0.3333 0. 0.]]
```

첫번째 클래스(Bream)에 대한 확률
↓
첫번째 샘플 → [0, 0, 0.6667, 0, 0.3333, 0, 0]
↑
두번째 클래스(Parkki)에 대한 확률

로지스틱 회귀

$$z = a \times \text{무게} + b \times \text{길이} + c \times \text{대각선} + d \times \text{높이} + e \times \text{두께} + f$$

$$\phi = \frac{1}{1 + e^{-z}}$$



로지스틱 회귀(이진 분류)

```
bream_smelt_indexes = (train_target == 'Bream') | (train_target == 'Smelt')
train_bream_smelt = train_scaled[bream_smelt_indexes]
target_bream_smelt = train_target[bream_smelt_indexes]
```

```
from sklearn.linear_model import LogisticRegression
```

```
lr = LogisticRegression()
lr.fit(train_bream_smelt, target_bream_smelt)
```

```
print(lr.predict(train_bream_smelt[:5]))
['Bream' 'Smelt' 'Bream' 'Bream' 'Bream']
```

```
print(lr.predict_proba(train_bream_smelt[:5]))
[[0.99759855 0.00240145]
 [0.02735183 0.97264817]
 [0.99486072 0.00513928]
 [0.98584202 0.01415798]
 [0.99767269 0.00232731]]
```

로지스틱 회귀(다중 분류)

```
lr = LogisticRegression(C=20, max_iter=1000)
lr.fit(train_scaled, train_target)
```

```
print(lr.score(train_scaled, train_target))
print(lr.score(test_scaled, test_target))
0.9327731092436975
0.925
```

```
proba = lr.predict_proba(test_scaled[:5])
print(np.round(proba, decimals=3))
[[0.      0.014 0.841 0.      0.136 0.007 0.003]
 [0.      0.003 0.044 0.      0.007 0.946 0.     ]
 [0.      0.      0.034 0.935 0.015 0.016 0.     ]
 [0.011 0.034 0.306 0.007 0.567 0.      0.076]
 [0.      0.      0.904 0.002 0.089 0.002 0.001]]
```

```
print(lr.coef_.shape, lr.intercept_.shape)
(7, 5) (7,)
```


감사합니다